Exercise 9 : Design input forms that validate data (e.g., email, phone number) and display error messages using HTML/CSS, JavaScript (with Validator.js)

AIM:

The aim is to design input forms that validate data, such as email and phone number, and display error messages using HTML/CSS and JavaScript with Validator.js. Validator.js includes validation functions such as isEmail() and isMobilePhone() that are commonly used for client-side form checks.

PROCEDURE:

Step 1:

Setting Up the HTML Form
Start by creating an HTML form with input fields for the email and phone number. The form should also include areas to display validation error messages near each input field so users can immediately see what needs to be corrected. Clear field-level feedback is a standard client-side validation pattern.

index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form Validation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="form-box">
    <h2>User Registration</h2>
    <form id="registerForm" novalidate>

      <div class="input-group">
        <label for="fullname">Full Name</label>
        <input type="text" id="fullname" placeholder="Enter your full name">
        <small class="error" id="nameError"></small>
      </div>

      <div class="input-group">
        <label for="email">Email Address</label>
        <input type="text" id="email" placeholder="Enter your email">
        <small class="error" id="emailError"></small>
      </div>

      <div class="input-group">
        <label for="phone">Phone Number</label>
        <input type="text" id="phone" placeholder="Enter your phone number">
        <small class="error" id="phoneError"></small>
      </div>
```

Step 2:

Styling the Form with CSS

Next, add basic styling using CSS to make the form visually neat and easy to use. Style the container, labels, input fields, buttons, and error messages so that the form is readable and user-friendly. Distinct visual feedback helps users recognize errors more quickly.

style.css > .invalid

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Segoe UI", sans-serif;
}

body {
  background: linear-gradient(135deg, #dbeafe, #eff6ff);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.form-box {
  background: #ffffff;
  width: 380px;
  padding: 30px;
  border-radius: 15px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.12);
}

.form-box h2 {
  text-align: center;
  margin-bottom: 20px;
  color: #1e3a8a;
}

.input-group {
  margin-bottom: 16px;
}
```

Step 3:

Adding JavaScript for Validation

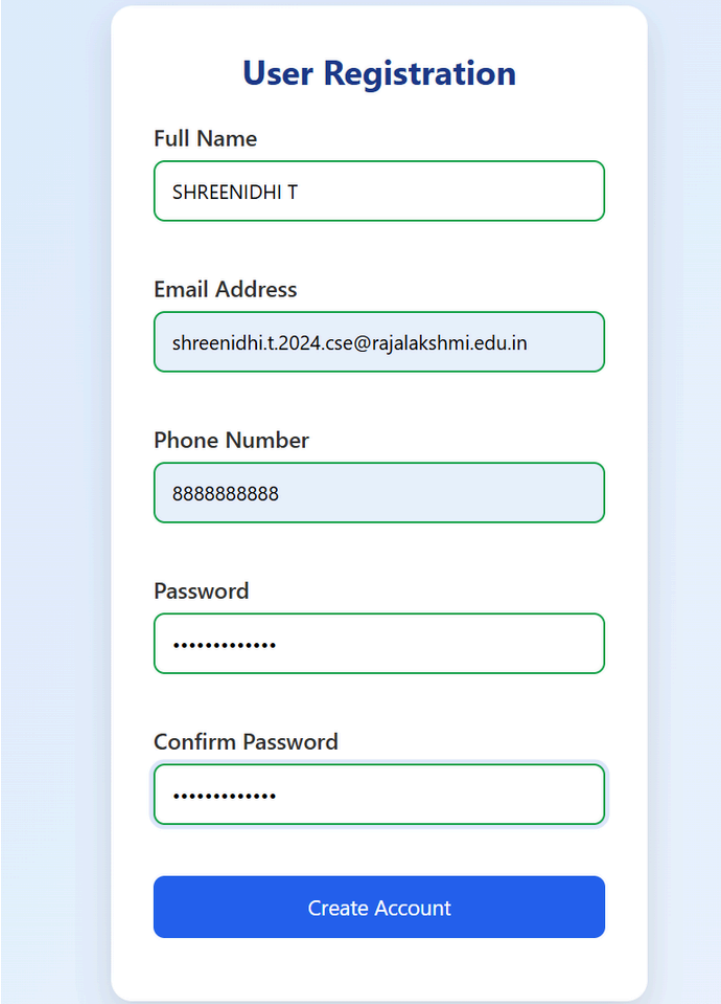
Finally, add JavaScript to validate the input fields using Validator.js and display error messages. The email field can be validated using isEmail(), while the phone number can be checked using isMobilePhone() with a suitable locale such as en-IN or any. Validator.js supports both methods for browser-based string validation.

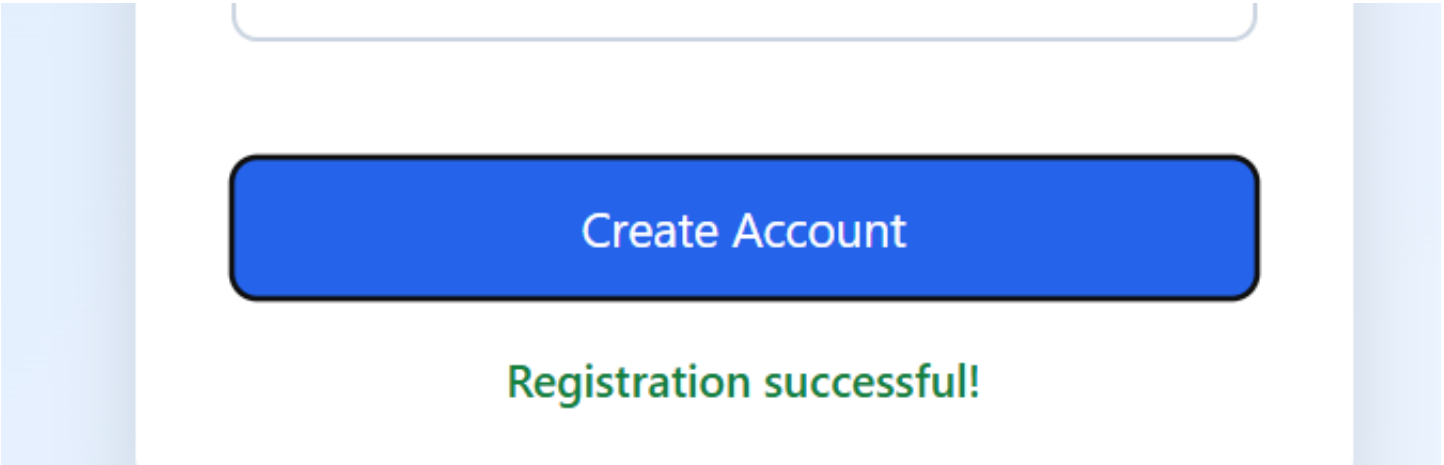
```javascript
const form = document.getElementById("registerForm");

const fullname = document.getElementById("fullname");
const email = document.getElementById("email");
const phone = document.getElementById("phone");
const password = document.getElementById("password");
const confirmPassword = document.getElementById("confirmPassword")

const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const phoneError = document.getElementById("phoneError");
const passwordError = document.getElementById("passwordError");
const confirmPasswordError = document.getElementById("confirmPass
const successMsg = document.getElementById("successMsg");

function setError(input, errorElement, message) {
  input.classList.add("invalid");
  input.classList.remove("valid");
  errorElement.textContent = message;
}

function setSuccess(input, errorElement) {
  input.classList.remove("invalid");
  input.classList.add("valid");
  errorElement.textContent = "";
}

function validateName() {
  const value = fullname.value.trim();
  if (value === "") {
    setError(fullname, nameError, "Full name is required.");
    return false;
  }
```

OUTPUT

For valid email and phone no.:

- The form accepts the entered values successfully.
- No error message is displayed.
- A success message such as **“Form submitted successfully”** can be shown. Client-side validation is used to ensure the entered data matches the expected format before submission.





Error message for invalid email:

- **Please enter a valid email address.**
  This message is displayed when the entered email does not follow a valid email format recognized by isEmail().

User Registration

Full Name

SHREENIDHI T

Email Address

shreenidhi.t.2024.cse@rajalakshmi.edu.i

Enter a valid email address.

Phone Number

8888888888